

VLSI

A report submitted in partial fulfillment of the
requirement for the award of the degree of

Bachelor of Technology in Electronics And Communication Engineering

by
DASARI HARSHNI
(Y22EC035)



Department of Electronics & Communication Engineering
R.V.R. & J.C. COLLEGE OF ENGINEERING
(Autonomous)

Approved by AICTE :: Affiliated to Acharya Nagarjuna University
Chowdavaram, Guntur - 522019, Andhra Pradesh, India

2025

Department of Electronics And Communication Engineering



CERTIFICATE

This is to certify that the report of **EC451 Internship** entitled "**VLSP**" that is being submitted by **DASARI HARSHINI(Y22EC035)** in partial fulfillment of the requirement of the Degree of **Bachelor of Technology in Electronics And Communication Engineering** to the R.V.R. & J.C. College of Engineering is a record of bonafide work carried out by him/her under my supervision.

Date: 20-JUNE-2025

Signature of Coordinator

Dr.X,Ascar Davix M.E., Ph.D

Associate Professor in ECE

Signature of HOD

Dr.T.Ranga Babu M.Tech., Ph.D

Professor & Head

Abstract

Very Large-Scale Integration (VLSI) is the process of designing and fabricating integrated circuits (ICs) with millions to billions of transistors on a single chip. Today, VLSI is at the heart of almost every electronic device, from smartphones, laptops, and IoT devices to supercomputers and AI accelerators.

With advancements in semiconductor technology, fabrication has moved to nanometer-scale processes (7nm, 5nm, and even 3nm nodes), enabling chips that are faster, smaller, more power-efficient, and more reliable. Modern VLSI design integrates complete systems into a single chip, known as a System-on-Chip (SoC), which combines processors, memory, communication units, and sensors in one package.

The field is supported by powerful Electronic Design Automation (EDA) tools and hardware description languages like Verilog and VHDL, which simplify the design process. Applications of VLSI extend to 5G/6G communication, artificial intelligence, autonomous vehicles, robotics, aerospace, and medical devices.

In today's world, VLSI is not just about miniaturization—it is about high performance, low power consumption, cost efficiency, and scalability, making it one of the most critical technologies driving the digital age.

Table of contents

Abstract	i
Table of Contents	ii
1 INTRODUCTION	1
1 INTRODUCTION TO VLSI	1
2 HISTORY	1
3 VLSI DESIGN FLOW	2
4 TYPES OF DIGITAL SYSTEMS/CIRCUITS	5
2 VERILOG	6
2.1 STRUCTURE:	7
2.2 DATA TYPE:	7
2.3 OPERATORS:	7
3 COMBINATIONAL CIRCUITS USING VERILOG	8
4 FINITE STATE MACHINES	20
5 HARDWARE DESCRIPTION LANGUAGE	24
6 FINAL PROJECT	26
6.1 PROJECT:N-BIT MULTIPLIER WITH BINARY TO BCD CON- VERTER	27
6.2 SHIFT AND ADD ALGORITHM	27
6.3 DOUBLE DABBLE ALGORITHM	28
6.4 FLOW CHART	29
6.5 VERILOG CODE	30
6.6 TESTBENCH CODE	32
7 CONCLUSION	35
8 REFERENCES	36

List of Figures

1.1	EVOLUTION	2
1.2	Y-CHART	3
1.3	FLOW CHART	4
3.1	FULL ADDER SCHEMATIC	9
3.2	FULL ADDER SIMULATION	9
3.3	RIPPLE CARRY ADDER SCHEMATIC	10
3.4	RIPPLE CARRY ADDER SIMULATION	10
3.5	4X1 MUX	11
3.6	4X1 MUX SIMULATION	12
3.7	1X4 DEMUX	13
3.8	1X4 DEMUX SIMULATION	13
3.9	8:3 ENCODER	15
3.10	8:3 ENCODER SIMULATION	15
3.11	3:8 DECODER	17
3.12	3:8 DECODER SIMULATION	18
3.13	GRAY TO BINARY	19
3.14	GRAY TO BINARY SIMULATION	19
4.1	MELAY STATE MACHINE	21
4.2	MELAY STATE DIAGRAM	21
4.3	MOORE DIAGRAM	23
4.4	MOORE STATE DIAGRAM	23
6.1	SHIFT AND ADD	27
6.2	DOUBLE DABBLE	28
6.3	FLOW CHART	29
6.4	OUTPUT WAVEFORMS	34
8.1	CERTIFICATE	37

1

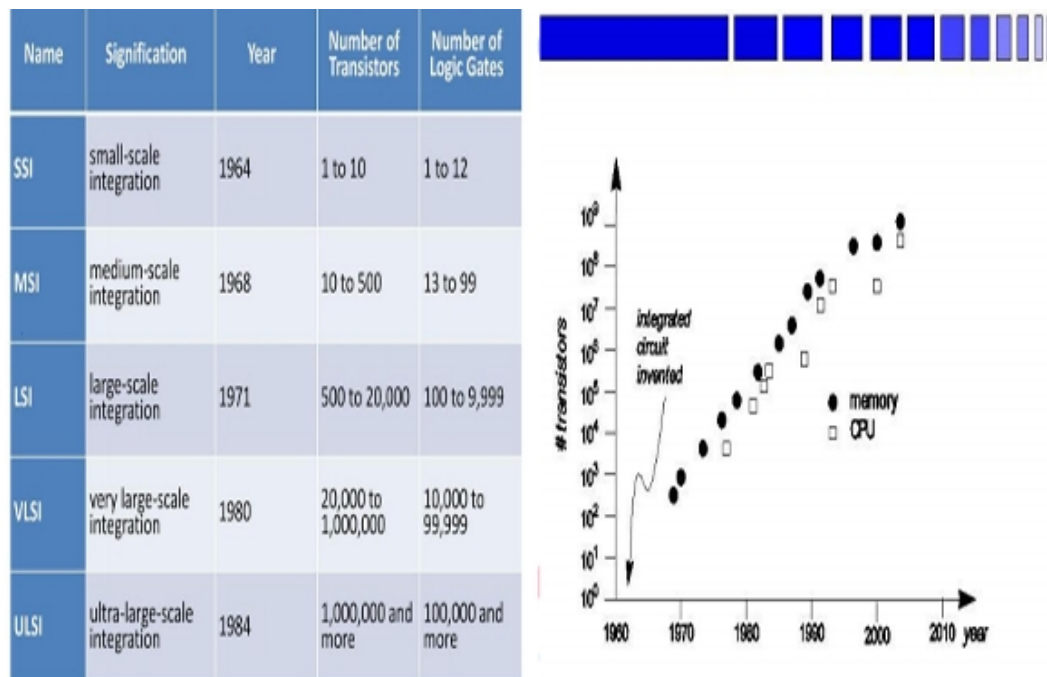
INTRODUCTION

1 INTRODUCTION TO VLSI

Very-large-scale integration (VLSI) is the process of creating an integrated circuit (IC) by combining thousands of transistors into a single chip. VLSI began in the 1970s when complex semiconductor and communication technologies were being developed. The microprocessor is a VLSI device. Before the introduction of VLSI technology, most ICs had a limited set of functions they could perform. An electronic circuit might consist of a CPU, ROM, RAM and other glue logic. VLSI lets IC designers add all of these into one chip. The electronics industry has achieved a phenomenal growth over the last few decades, mainly due to the rapid advances in large scale integration technologies and system design applications. With the advent of very large scale integration (VLSI) designs, the number of applications of integrated circuits (ICs) in high-performance computing, controls, telecommunications, image and video processing, and consumer electronics has been rising at a very fast pace. The current cutting-edge technologies such as high resolution and low bit-rate video and cellular communications provide the end-users a marvellous number of applications, processing power and portability. This trend is expected to grow rapidly, with very important implications on VLSI design and systems design

2 HISTORY

Circuits are tiny electronic circuit used to perform a specific electronic function. The first integrated Circuit (IC) was invented by Jack Kilby in 1958. As suggested by Moore, the capacity doubled roughly every 18 months. Today, a large single VLSI chip can contain

Figure 1.1: **EVOLUTION**

over one billion transistors. These days, VLSI chiefly comprises of Front End Design and Back End design. Front end design includes digital design using HDL and design verification through simulation and other techniques. The backend design comprises of CMOSlibrary design and its characterization. It also covers the physical design and fault simulation

In April 1965, one of the co-founders of Intel, Dr. Moore, predicted that the number of components in an integrated circuit would double every year. Ten years later in 1975, he revised his prediction to a doubling of every 2 years. Moore's prediction, which is commonly known as Moore's law nowadays, has been widely used in the semiconductor and microelectronic industries as a tool to predict the increase of components in a chip for the coming generations.

3 VLSI DESIGN FLOW

The design process starts with a given set of requirements. Initial design is developed and tested against the requirements. When requirements are not met, the design has to be improved. If such improvement is either not possible or, then the revision of requirements and its impact analysis must be considered. The Y-chart shown in Fig. 1.4 illustrates a design flow for most logic chips, using design activities on three different axes (domains) which resemble the letter Y. The Y-chart consists of three major domains, namely: behavioural domain. structural domain. geometrical layout domain. The design flow starts from the algorithm that describes the behavior of the target chip. The corresponding architecture of the processor is first defined. The next design evolution in the behavioural domain defines finite state machines (FSMs) which are structurally implemented with functional modules such as registers and ALUs. These modules are then geometrically placed onto the chip surface using CAD tools for automatic module placement followed by routing, with a goal of minimizing the interconnects area and signal delays. The third evolution starts with a

behavioral module description. Individual modules are then implemented with leaf cells. At this stage the chip is described in terms of logic gates (leaf cells), which can be placed and interconnected by using a cell placement routing program. The last evolution involves a detailed Boolean description of leaf cells followed by a transistor level implementation of leaf cells and mask generation. In standard cell based design, leaf cells are already pre-designed and stored in a library for logic de-IFICATION AND ARCHITECTURAL DESIGN VLSI design cycle start with a formal specification of a VLSI chip. At this stage, the system specifications are laid out. FUNCTION DESIGN In this stage, main functional units of the system and the interconnect requirements between the units are identified. The main purpose of this stage is to specify system behavior, in terms of Input, Output, and Timing of each unit. LOGIC DESIGN In this stage, the logic for the VLSI system is designed. It includes, Boolean expressions, control flow, register allocation, etc. The outcome is a RTL description that is expressed in a Hardware Description Language (HDL) like VHDL and Verilog. CIRCUIT DESIGN The purpose of the circuit design is to develop a circuit representation based on the logic design. FABRICATION Fabrication process includes lithography, polishing, deposition, diffusion, etc. This process consists of several steps and requires various masks. PACKAGING Pack aging involves putting together the chips on a Printed Circuit Board (PCB) or a Multi-Chip. Your paragraph text goes here... It will appear above the image in the PDF.

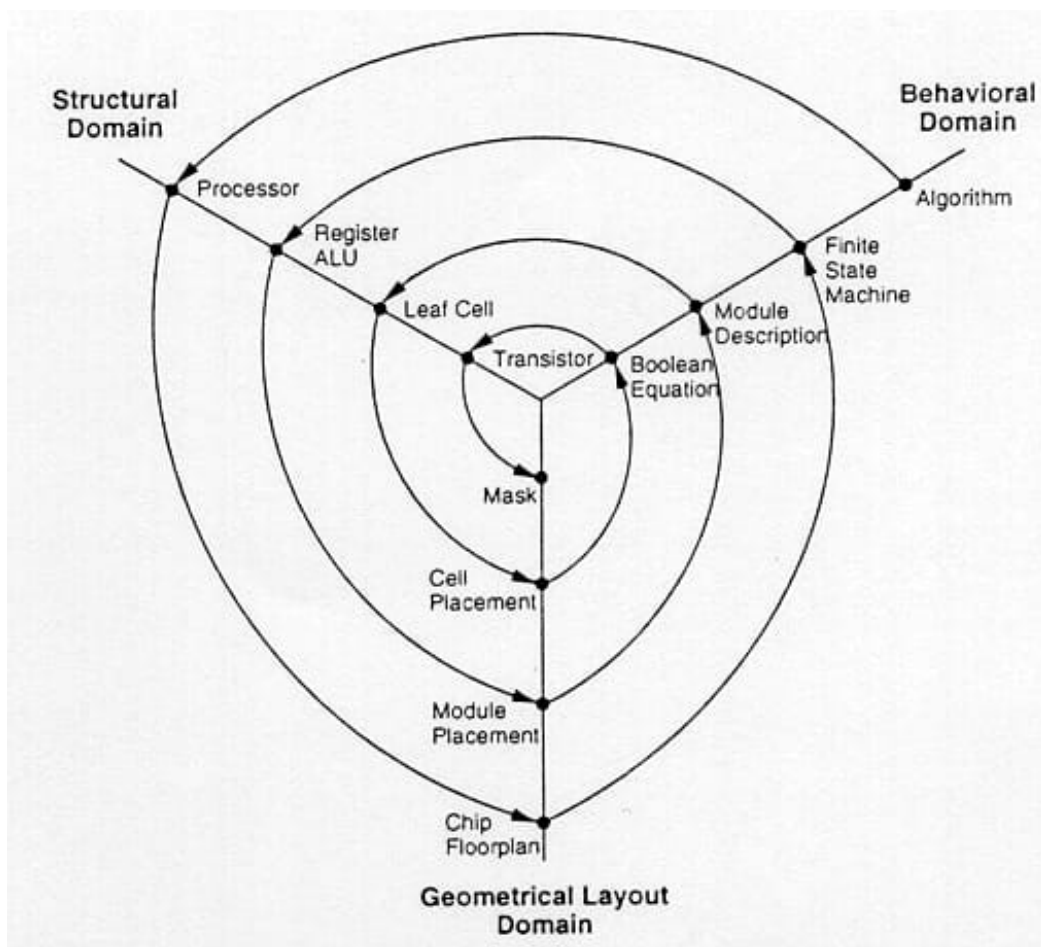


Figure 1.2: Y-CHART

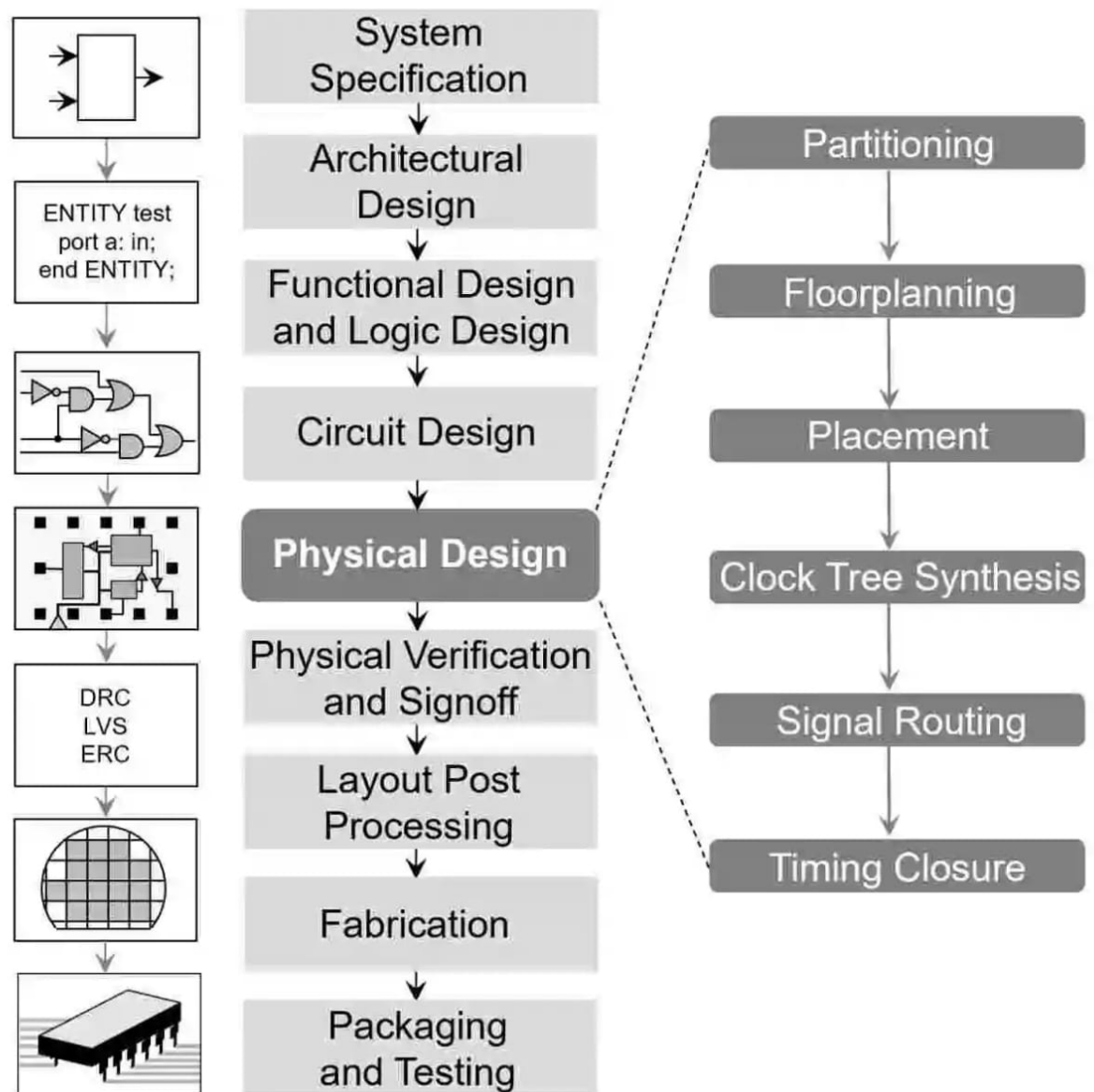


Figure 1.3: FLOW CHART

4 TYPES OF DIGITAL SYSTEMS/CIRCUITS

A digital logic circuit is defined as the one in which voltages are assumed to be having a finite number of distinct values. Types of digital logic circuits are combinational logic circuits and sequential logic circuits. These are the basic circuits used in most of the digital electronic devices like computers, calculators, mobile phones. Combinational logic circuits are basically made up of digital logic gates like AND gate, OR gate, NOT gate and universal gates (NAND gate and NOR gate). All these gates are combined together to form a complicated switching circuit. In a combinational logic circuit, the output at any instant of time depends only on present input at that particular instant of time and combinational circuits do not have any memory devices. Combinational digital logic circuits are classified into three major parts— arithmetic or logical functions, data transmission and code converter. Sequential logic circuits A Sequential digital logic circuit is different from combinational logic circuits. In sequential circuit the output of the logic device is not only dependent on the present inputs to the device, but also on past inputs. Unlike combinational circuits, the sequential circuits have memory devices in order to store the past outputs. In fact, sequential digital logic circuits are nothing but combinational circuit with memory. These types of digital logic circuits are designed using finite state machine. Types of sequential logic circuits Sequential digital circuits are classified into three major parts as Event driven, Clock drive and Pulse driven

2

VERILOG

It is a hardware description language used for the design and analysis of digital circuits. The syntax and structure of Verilog is similar to that of the C programming language. It is case sensitive. Modules are the basic building blocks of the Verilog code. Verilog is also more compact since the language is more of an actual hardware modeling language.

2.1 STRUCTURE:

Module name always start with an alphabet or underscore. It containing three parts namely:

- o Module declaration: We declare the module name, input and outputs.
- o Body of the module: Here we can assign the relationship between the inputs outputs.
- o End of the module: It is the terminating of the module or the code. Keyword “assign” is used to assign the parameters.

2.2 DATA TYPE:

Integer: It is declared by predefined word “integer”. Real: It is used to declare floating point numbers. Net:

- o Used to declare the physical connections between the entities.
- o Used when the signal is driven by the output of some device.
- o They have continuously changing value by the circuits that are driving them.
- o They can take the values 0, 1, X(don't care), Z(High impedance)

o Net datatype can be declared by the predefined word using “wire”. Ex: wire w1, w2

Registers: Registers store the values until they are updated.

Vectors: They are used to declare ‘n’ bit values. Register or net can be declared as vectors. For ex,,,,, to declare total as a register as a 8 bits whose initial value is decimal 12. reg [7:0] total=8'd12, where ‘d’ stands for decimal. Parameter: It represents global constants and is declared by predefined word “parameter”. o Example: module.....; parameter m=3; o Here m is a constant value 3. Its value remains same throughout the program. Array: Registers and integers can be written as arrays. It is used to store the collection of similar type of values. Example: integer [4:0] total [0:3]; o Here total is an array of 4 elements and each element has an integer of size 5 bit.

2.3 OPERATORS:

Arithmetic Operators, Relational Operators, Bitwise Operators, Logical Operators, Reduction Operators, Shift Operators, Concatenation Operators, Replication Operators, Conditional Operators.

3

COMBINATIONAL CIRCUITS USING VERILOG

A Combinational Circuit consist of logic gates whose outputs at any instant of time are determined directly from the present combination of inputs without regard to previous input. Examples of combinational circuits: Adder, Subtractor, Converter, and Encoder/Decoder.

1) Full Adder Design: Full Adder is the adder that adds three inputs and produces two outputs. The first two inputs are A and B and the third input is an input carry as C-IN. The output carry is designated as COUT and the normal output is designated as S which is SUM.

```
module FA\_gate (  
    input a,input b,input c,  
    output sum, output cout);  
    wire s1, s2, s3;  
    xor x1 (s1, a, b);  
    and a1 (s2, a, b);}  
    xor x2 (sum, s1, c);
```

```

and a2 (s3, s1, c);
or o1 (cout, s2, s3);
endmodule

```

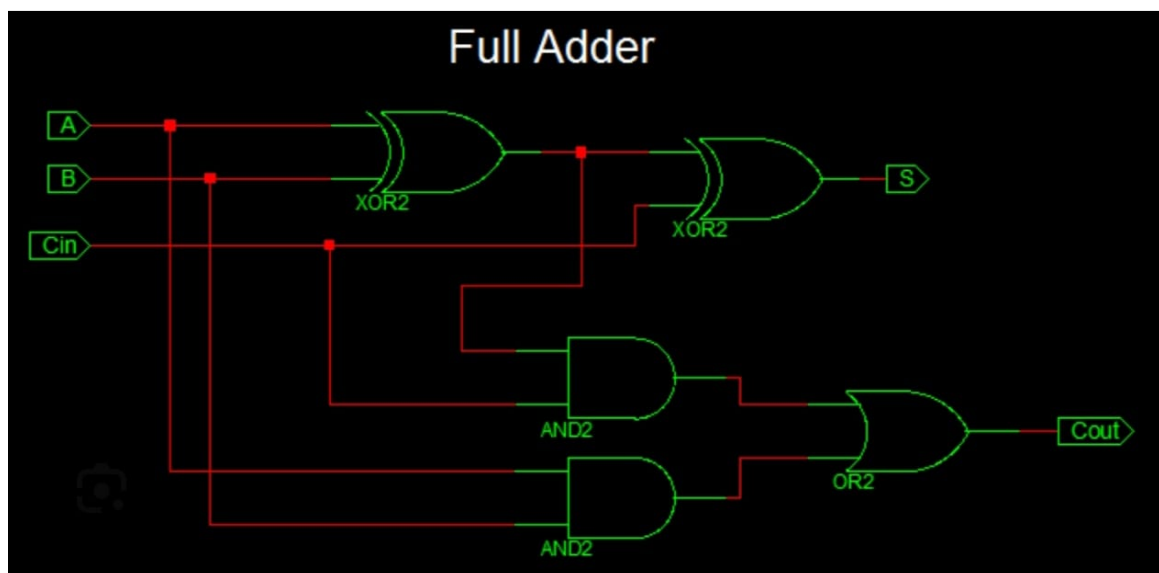


Figure 3.1: FULL ADDER SCHEMATIC



Figure 3.2: FULL ADDER SIMULATION

2) 4-bit ripple carry adder 4-bit ripple carry adder is used for the purpose of adding two 4-bit binary numbers. Ripple Carry Adder works in different stages. Each full adder takes the carry-in as input and produces carry-out and sum bit as output.

```

module RPA\_4bit (
    input [3:0] a, input [3:0] b,
    input cin,
    output [3:0] s, output cout);
    wire c1, c2, c3;
    FA\_if d0 (a[0], b[0], c, s[0], c1);
    FA\_if d1 (a[1], b[1], c1, s[1], c2);
    FA\_if d2 (a[2], b[2], c2, s[2], c3);
    FA\_if d3 (a[3], b[3], c3, s[3], cout);
endmodule

```

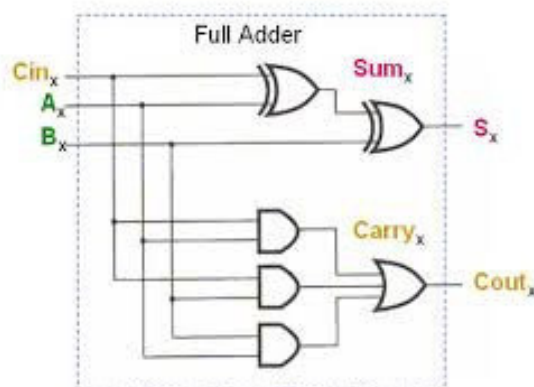


Figure 3.3: **RIPPLE CARRY ADDER SCHEMATIC**

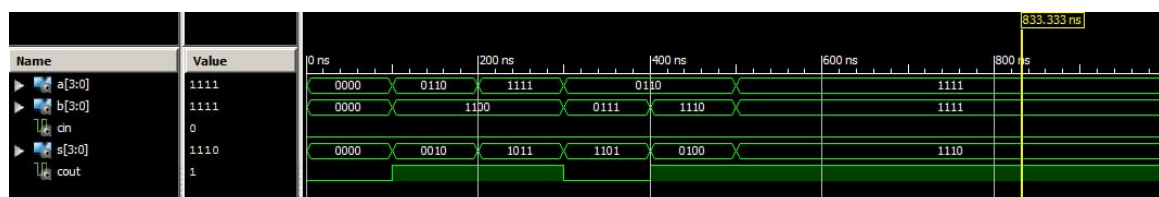


Figure 3.4: **RIPPLE CARRY ADDER SIMULATION**

3) 4x1 MUX Design 4x1 Multiplexer has four data inputs D0, D1, D2 D3, two selection lines S0 S1 and one output Y. The block diagram of 4x1 Multiplexer is shown in the following figure. One of these 4 inputs will be connected to the output based on the combination of inputs present at these two selection lines.

```

module MUX4x1(
input i0, input i1, input i2,
input i3, input s1, input s0,
output y);
wire p,q;
MUX\_2x1\_gate d0 (i0, i1, s0, p);
MUX\_2x1\_gate d1 (i2, i3, s0, q);
MUX\_2x1\_gate d2 (p, q, s1, y);
endmodule

```

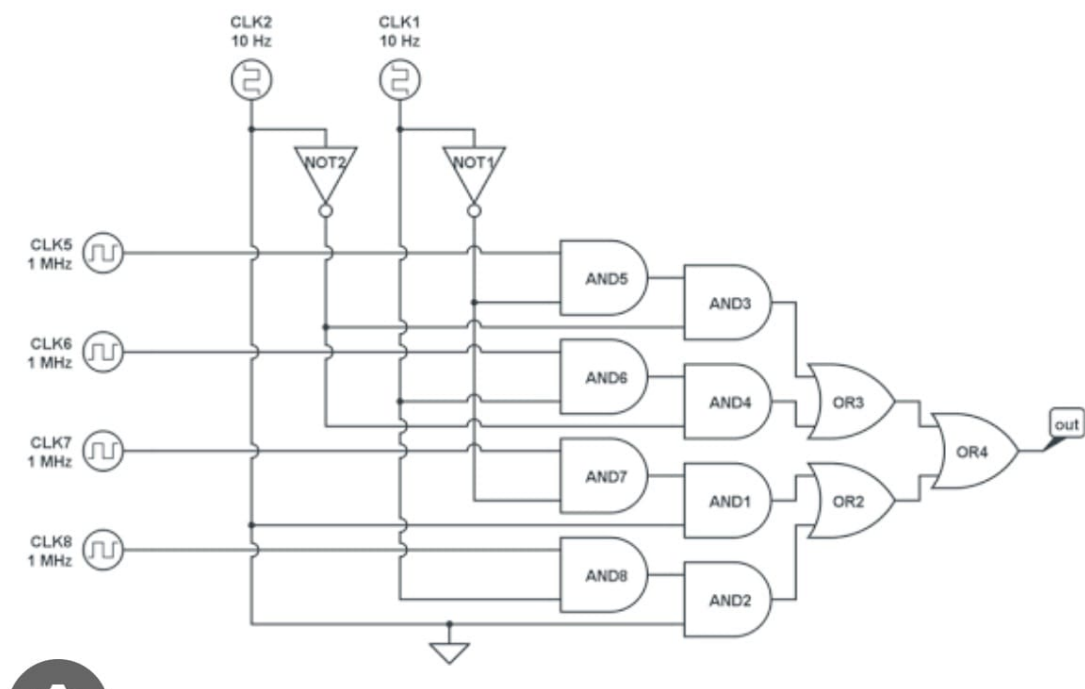


Figure 3.5: 4X1 MUX

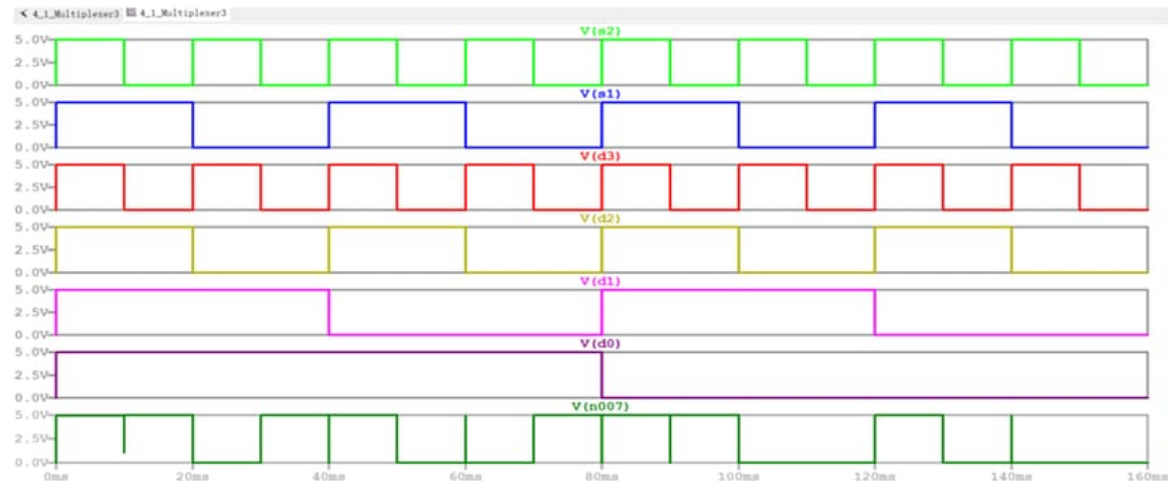


Figure 3.6: 4X1 MUX SIMULATION

4) 1x4 DEMUX Design A 1-to-4 demultiplexer has a single input (D), two selection lines (S1 and S0) and four outputs (Y0 to Y3). The input data goes to any one of the four outputs at a given time for a particular combination of select line

```
module DEMUX1x4 (
    input x, input s1, input s0,
    output y0, output y1, output y2,
    output y3);
    assign y0 = x & (~s1) & (~s0);
    assign y1 = x & (~s1) & (s0);
    assign y2 = x & (s1) & (~s0);
    assign y3 = x & (s1) & (s0);
endmodule
```

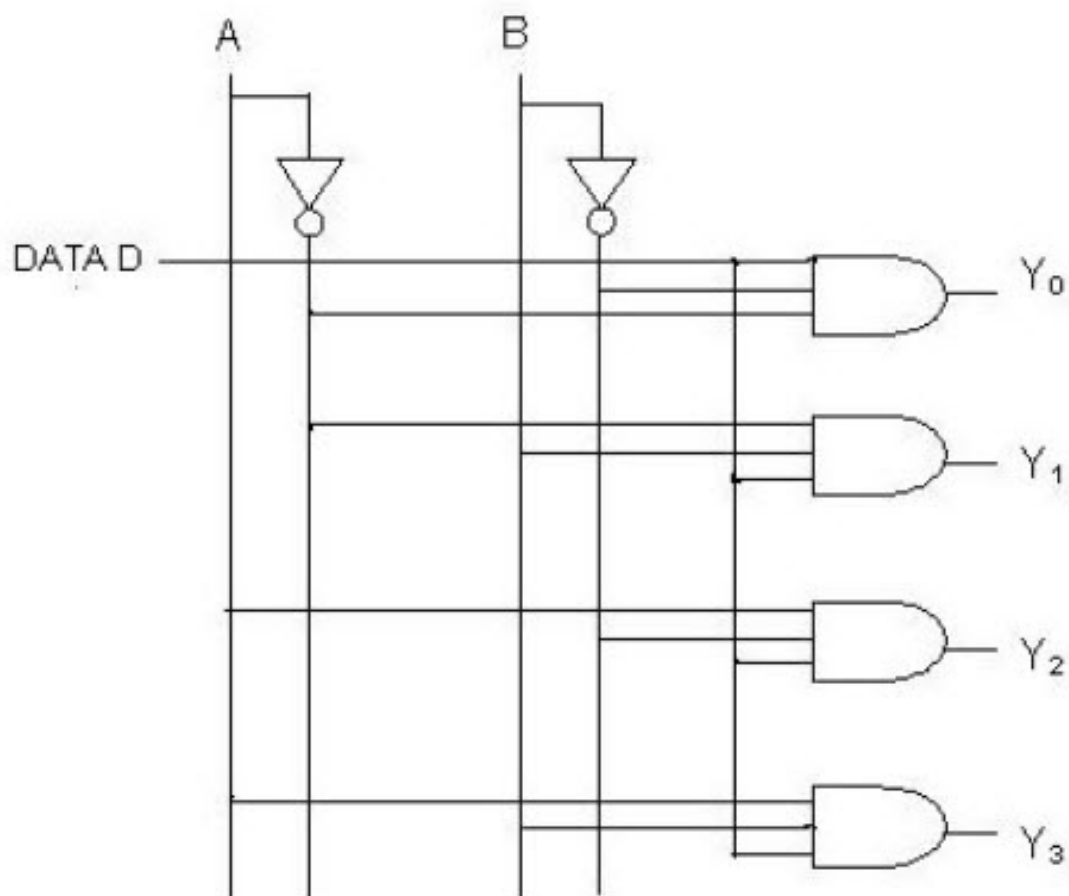


Figure 3.7: 1X4 DEMUX

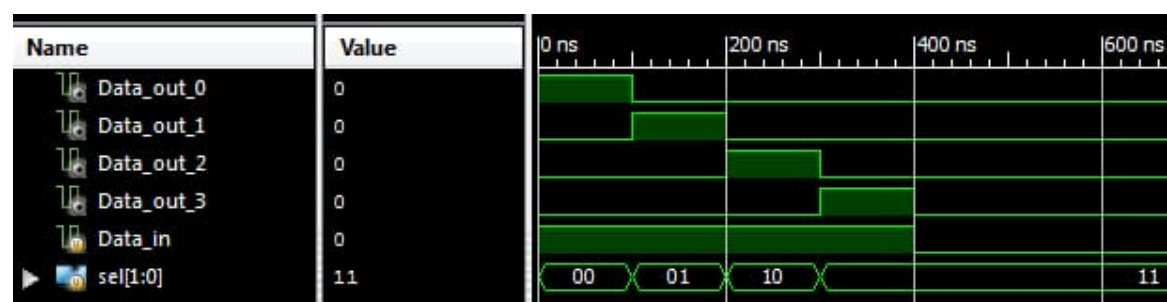


Figure 3.8: 1X4 DEMUX SIMULATION

5) 8:3 Encoder The 8:3 Encoder is also known as Octal to Binary Encoder. In 8:3 encoder, there is a total of eight inputs, i.e., Y0, Y1, Y2, Y3, Y4, Y5, Y6, and Y7 and three outputs, i.e., A0, A1, and A2. In 8-input lines, one input-line is set to true at a time to get

the respective binary code in the output side

```
module encoder\_8to3(  
    input [7:0] y, input enable,  
    output reg [2:0] a);  
    always @(*) begin  
        if (enable == 0)  
            a = 8'b0; else begin  
                case (d)  
                    8'b00000001 : a = 3'y0;  
                    8'b00000010 : a = 3'y1;  
                    8'b00000100 : a = 3'y2;  
                    8'b00001000 : a = 3'y3;  
                    8'b00010000 : a = 3'y4;  
                    8'b00100000 : a = 3'y5;  
                    8'b01000000 : a = 3'y6;  
                    8'b10000000 : a = 3'y7;  
                    default : a = 3'y0;  
                endcase  
            end  
        end  
    endmodule
```

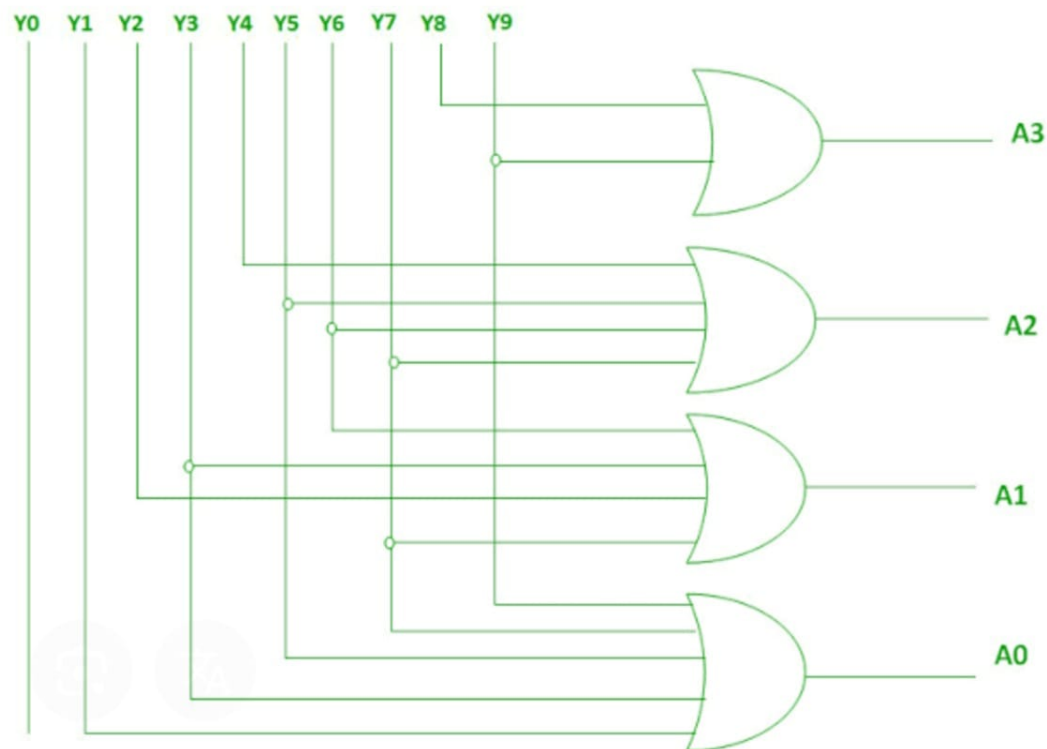


Figure 3.9: 8:3 ENCODER

Msgs							
Y	3'h3	3'h1		3'h2		3'h3	
	1'h0						
	1'h1						
	1'h1						
D	8'h08	8'h02	8'h03	8'h04	8'h05	8'h06	8'h07
	1'h0						
	1'h0						
	1'h0						
	1'h0						
	1'h1						
	1'h0						

Figure 3.10: 8:3 ENCODER SIMULATION

6)3:8 Decoder A3to 8decoder has three inputs (A, B, C) and eight outputs (D0 to D7). Based on the 3 inputs one of the eight outputs is selected. The truth table for 3 to 8 decoder is shown in the below table. From the truth table, it is seen that only one of eight outputs (D0 to D7) is selected based on three select inputs

```
module Dec3to8 (
```

```
input a, input b, input c,  
output y0, output y1, output y2,  
output y3, output y4, output y5,  
output y6, output y7, input E);  
wire p,q;  
Dec\_2to4\_En d0 (E, c,0,p,q);  
Dec\_2to4\_En d1 (p, a,b, y0,y1,y2,y3);  
Dec\_2to4\_En d2 (q, a,b,y4,y5,y6,y7);  
endmodule
```

Your paragraph text goes here... It will appear above the image in the PDF.

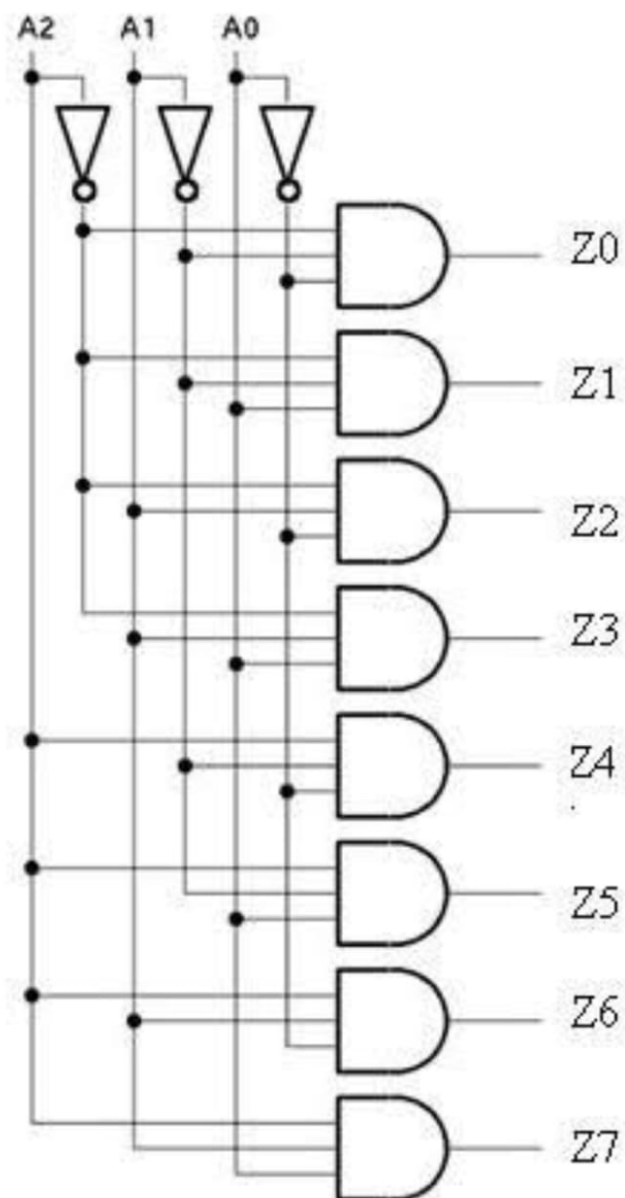


Figure 3.11: 3:8 DECODER

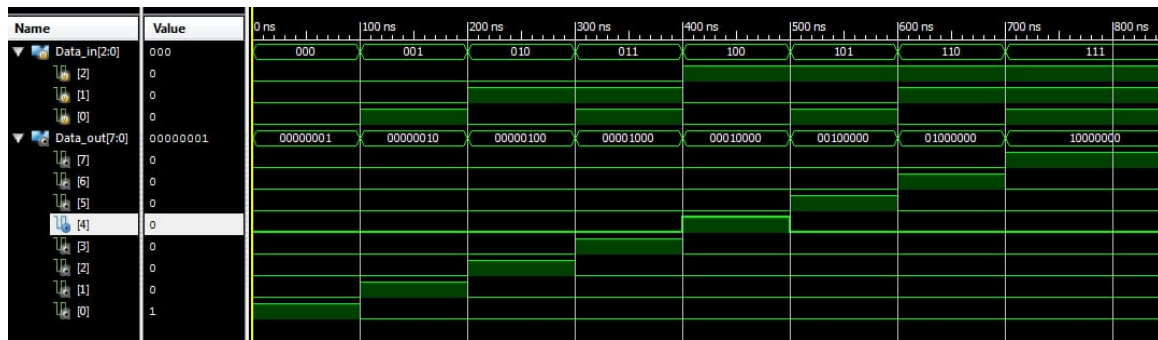


Figure 3.12: 3:8 DECODER SIMULATION

7) Code Converter (Gray to Binary) Gray Code system is a binary number system in which every successive pair of numbers differs in only one bit. It is used in applications in which the normal sequence of binary numbers generated by the hardware may produce an error or ambiguity during the transition from one number to the next. For ex, the states of a system may change from 3(011) to 4(100) as- 011 — 001 — 101 — 100. So, there is a high chance of a wrong state being read while the system changes from the initial state to the final state. This could have serious consequences for the machine using the information. The Gray code eliminates this problem since only one bit changes its value during any transition between two numbers

```

module GtoB\_data(
input [3:0] g, output [3:0] b);
assign b[3] = g[3];
assign b[2] = g[3] ^ g[2];
assign b[1] = g[3] ^ g[2] ^ g[1];
assign b[0] = g[3] ^ g[2] ^ g[1] ^ g[0];
endmodule

```

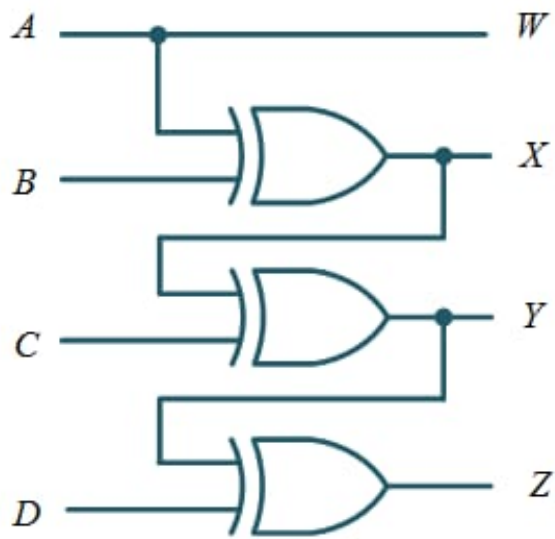


Figure 3.13: GRAY TO BINARY

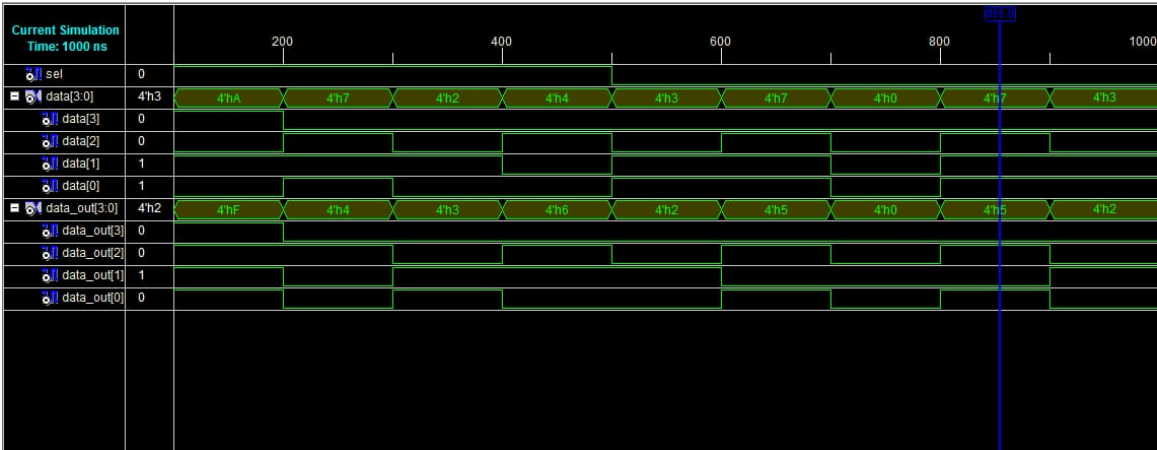


Figure 3.14: GRAY TO BINARY SIMULATION

4

FINITE STATE MACHINES

The Finite State Machine is an abstract mathematical model of a sequential logic function. It has finite inputs, outputs, and number of states. FSMs are implemented in real-life circuits through the use of Flip Flops. There are two types of FSMs: Mealy State Machine Moore State Machine

1) Mealy machine: A Finite State Machine is said to be Mealy state machine, if outputs depend on both present inputs present states. The block diagram of Mealy state machine is shown in the following figure. As shown in figure, there are two parts present in Mealy state machine. Those are combinational logic and memory. Memory is useful to provide some or part of previous outputs present states as inputs of combinational logic. So, based on the present inputs and present states, the Mealy state machine produces outputs. Therefore, the outputs will be valid only at positive or negative transition of the clock signal. In the above figure, there are three states, namely A, B C. These states are labelled inside the circles each circle corresponds to one state. Transitions between these states are represented with directed lines. Here, 0 / 0, 1 / 0 1 / 1 denotes input / output. In the above figure, there are two transitions from each state based on the value of input, x.

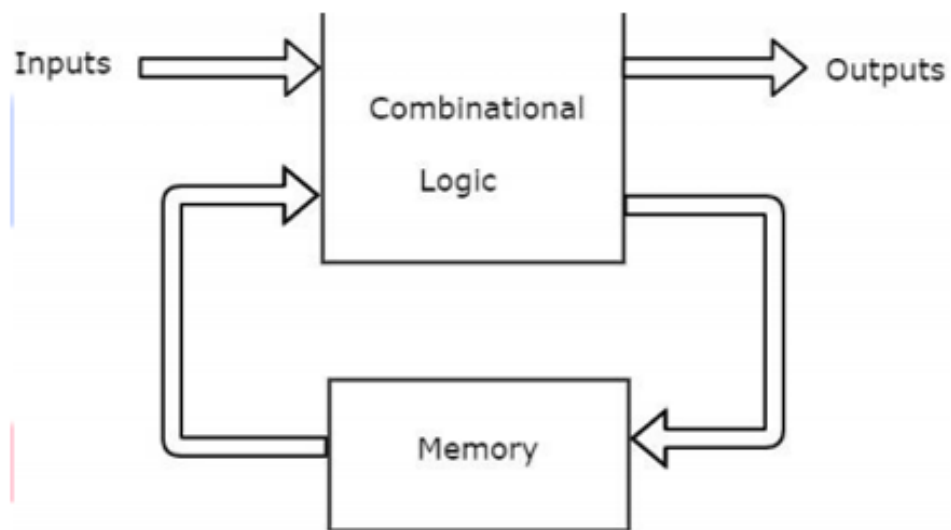


Figure 4.1: MELAY STATE MACHINE

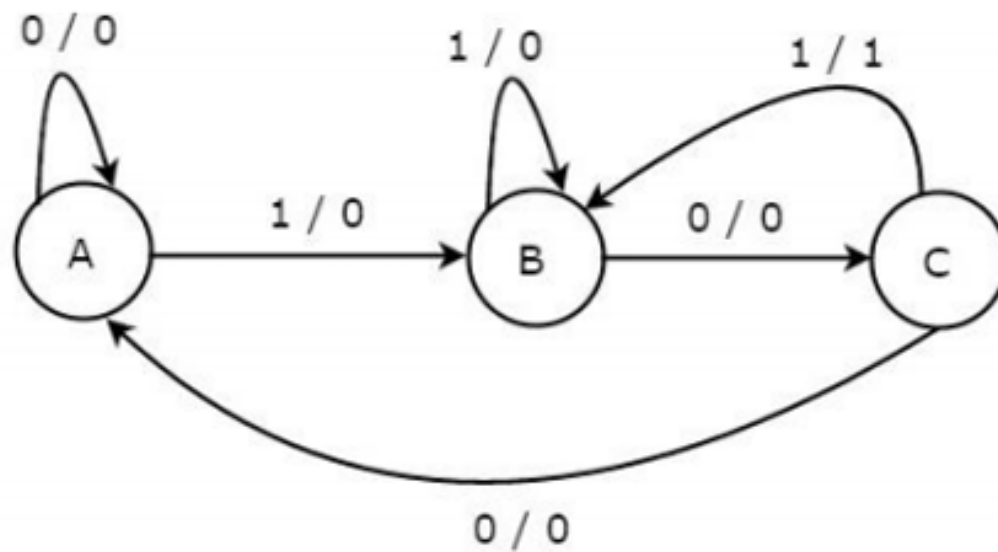


Figure 4.2: MELAY STATE DIAGRAM

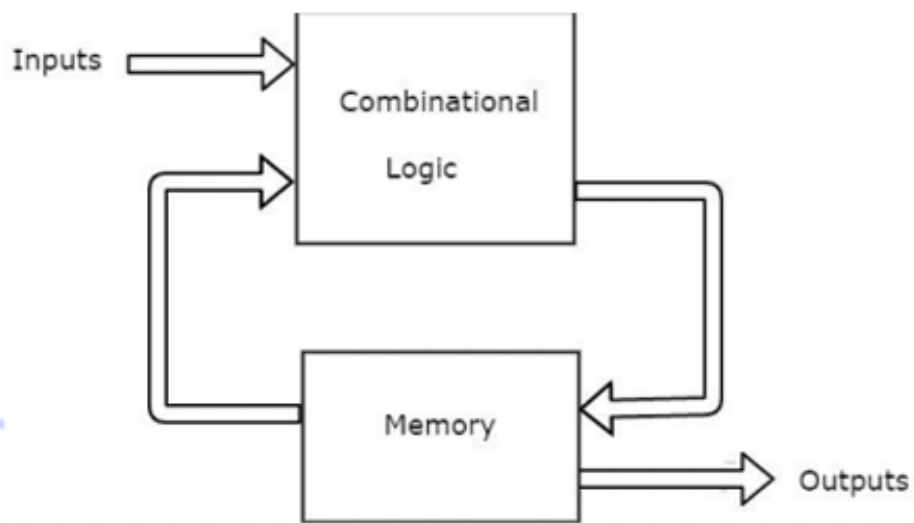
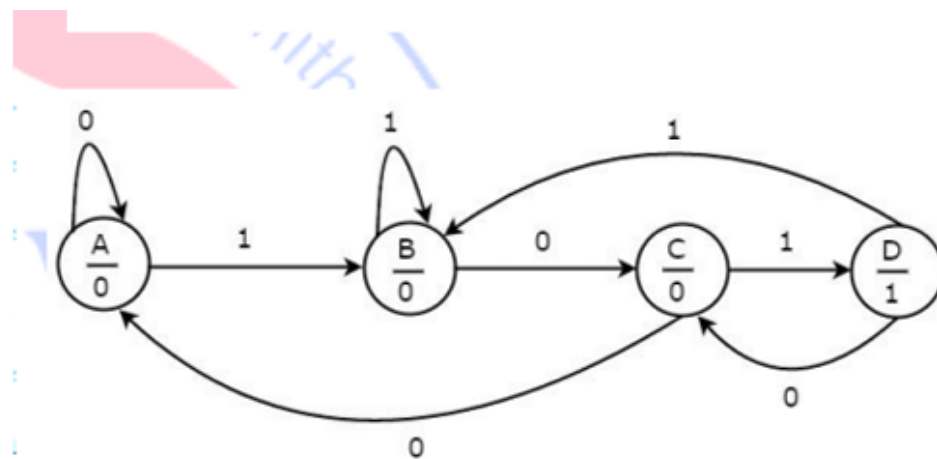
2) Mealy machine: A Finite State Machine is said to be Moore state machine, if outputs depend only on present states. The block diagram of Moore state machine is shown in the following figure. As shown in figure, there are two parts present in Moore state machine. Those are combinational logic and memory. In this case, the present inputs and present states determine the next states. So, based on next states, Moore state machine produces the outputs. Therefore, the outputs will be valid only after transition of the state. In the above figure, there are four states, namely A, B, C, D. These states and the respective outputs are labelled inside the circles. Here, only the input value is labeled on each transition. In the above figure, there are two transitions from each state based on the value of input, x

A Moore machine is a type of finite state machine (FSM) in which the output depends only on the present state and not directly on the input. It is formally defined as a 6-tuple $M = (Q, \Sigma, \Gamma, \delta, \lambda, q_0)$ where

Q is the set of states, Σ is the input alphabet, Γ is the output alphabet, δ is the state transition function, λ is the output function that maps states to outputs, and $q_0 \in Q$

q_0 is the initial state. In a Moore machine, the output is associated with the states, which means the output changes only when the system moves from one state to another. This property makes Moore machines more stable and less prone to glitches, though they may introduce a one-clock cycle delay in producing the output.

Compared to Mealy machines, Moore machines are easier to design and debug because the outputs depend solely on states, but they often require more states for the same functionality. On the other hand, Mealy machines can be faster and require fewer states since their output depends on both the state and the input. Despite these differences, Moore machines are widely used in applications such as sequence detectors, digital control systems, and protocol design in communication systems, where predictable and stable output behavior is important.

Figure 4.3: **MOORE DIAGRAM**Figure 4.4: **MOORE STATE DIAGRAM**

5

HARDWARE DESCRIPTION LANGUAGE

In the early days of Integrated Circuits, engineers had to sit down and physically draw transistors and their connections on paper to design them such that it can be fabricated on silicon. Bigger and complex circuits demanded more engineers, time and other resources and soon enough there was a need to have a better way of designing integrated circuits. Hardware description language (HDL) is a specialized computer language used to program electronic and digital logic circuits. The structure, operation and design of the circuits are programmable using HDL. HDL includes a textual description consisting of operators, expressions, statements, inputs and outputs. Instead of generating a computer executable file, the HDL compilers provide a gate map. The gate map obtained is then downloaded to the programming device to check the operations of the desired circuit. The language helps to describe any digital circuit in the form of structural, behavioural and gate level and it is found to be an excellent programming language for FPGAs and CPLDs. **The three common HDLs are Verilog, VHDL, and SystemC.** Of these, SystemC is the newest. The

HDLs will allow fast design and better verification. In most of the industries, Verilog and VHDL are common. Verilog, one of the main Hardware Description Language standardized as IEEE 1364 is used for designing all types of circuits. It consists of modules and the language allows Behavioral, Dataflow and Structural Description. VHDL (Very High Speed Integrated Circuit Hardware Description Language) is standardized by IEEE 1164. The design is composed of entities consisting of multiple architectures. SystemC is a language that consists a set of C++ classes and macros. It allows electronic system level and transaction modeling.

Need for Hardware Description Language: The Moore's Law in the year 1970 has brought a drastic change in the field of IC technology. This change has made the developers to bring out complex digital and electronic circuits. But the problem was the absence of a better programming language allowing hardware and software codesign. Complex digital circuit designs require more time for development, synthesis, simulation and debugging. The arrival of HDLs has helped to solve this problem by allowing each module to be worked by a separate team. All the goals like power, throughput, latency (delay), test coverage, functionality and area consumption required for a design can be known by using HDL. As a result, the designer can make the necessary engineering tradeoffs and can develop the design in a better and efficient way. Simple syntax, expressions, statements, concurrent and sequential programming is also necessary while describing the electronics circuits. All these features can be obtained by using a hardware description language

6

FINAL PROJECT

6.1 PROJECT:N-BIT MULTIPLIER WITH BINARY TO BCD CONVERTER

INTRODUCTION TO PROJECT: This project is designed as an n-bit multiplier of two numbers. The multiplication is performed using the shift and add method of multiplying two numbers.

Once the multiplication is completed, the output result is also converted to its binary coded decimal (BCD) representation.

This is done using the double dabble method of converting binary numbers to binary coded decimal(BCD)

In the output, both the binary and the binary-coded decimal (BCD) representation is shown.

6.2 SHIFT AND ADD ALGORITHM

Consider two numbers 29 and 9 Their binary representation is 11101 and 1001 The result is 261 Their binary representation is 100000101.

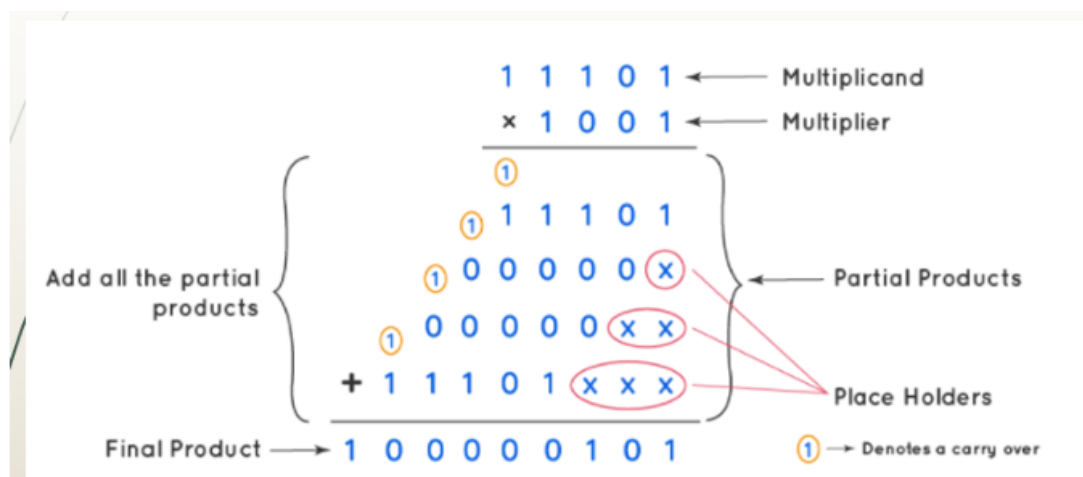


Figure 6.1: SHIFT AND ADD

6.3 DOUBLE DABBLE ALGORITHM

Consider the number 151. Its binary representation is 10010111

BCD2	BCD1	BCD0	Binary Input	
0000	0000	0000	10010111	
0000	0000	0001	0010111.	← ①
0000	0000	0010	010111..	← ②
0000	0000	0100	10111...	← ③
0000	0000	1001	0111....	← ④
0000	0000	1100	0111....	ADD
0000	0001	1000	111.....	← ⑤
0000	0001	1011	111.....	ADD
0000	0011	0111	11.....	← ⑥
0000	0011	1010	11.....	ADD
0000	0111	0101	1.....	← ⑦
0000	1010	1000	1.....	ADD ADD
0001	0101	0001		← ⑧

Figure 6.2: DOUBLE DABBLE

6.4 FLOW CHART

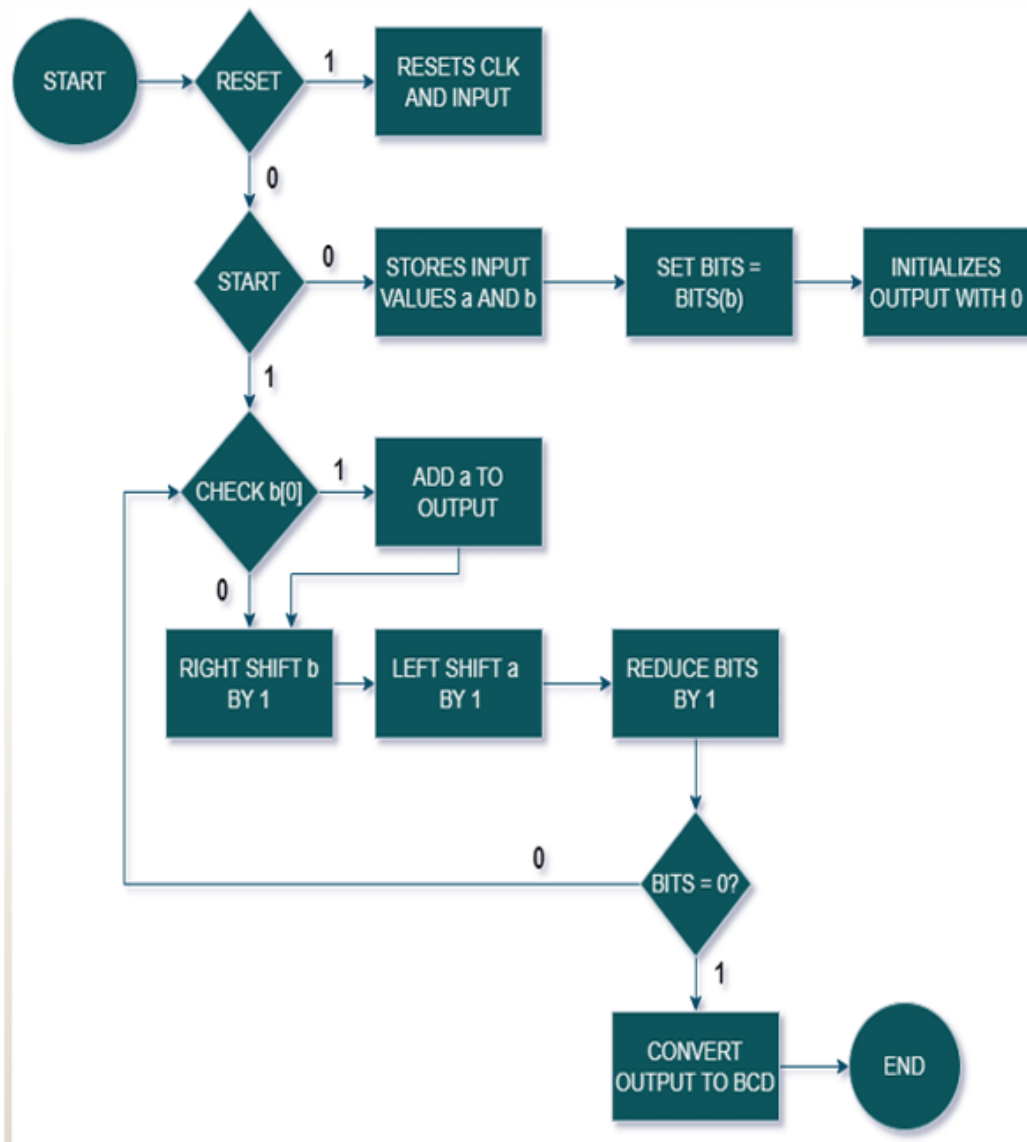


Figure 6.3: FLOW CHART

6.5 VERILOG CODE

```
//=====
// N-bit Multiplexer
//=====

module n_bit_mux #(parameter N = 8) (
    input  [N-1:0] in0, in1,    // two N-bit inputs
    input          sel,        // select line
    output [N-1:0] out          // N-bit output
);
    assign out = (sel) ? in1 : in0;
endmodule


//=====
// Binary to BCD Converter (Double Dabble Algorithm)
//=====

module binary_to_bcd #(parameter N = 8) (
    input  [N-1:0] binary,    // N-bit binary input
    output reg [3:0] ones,     // Ones place
    output reg [3:0] tens,     // Tens place
    output reg [3:0] hundreds // Hundreds place
);
    integer i;
    reg [N+11:0] shift_reg;    // Temporary register for shifting

    always @(*) begin
        // initialize
        shift_reg = 0;
```

```

    shift_reg[N-1:0] = binary;

    // double dabble process
    for(i=0; i<N; i=i+1) begin
        if (shift_reg[N+3:N] >= 5)
            shift_reg[N+3:N] = shift_reg[N+3:N] + 3;
        if (shift_reg[N+7:N+4] >= 5)
            shift_reg[N+7:N+4] = shift_reg[N+7:N+4] + 3;
        if (shift_reg[N+11:N+8] >= 5)
            shift_reg[N+11:N+8] = shift_reg[N+11:N+8] + 3;

        shift_reg = shift_reg << 1;
    end

    // assign results
    ones      = shift_reg[N+3:N];
    tens      = shift_reg[N+7:N+4];
    hundreds  = shift_reg[N+11:N+8];
end
endmodule

//=====
// Top Module: MUX + Binary-to-BCD Converter
//=====
module mux_with_bcd #(parameter N = 8) (
    input  [N-1:0] in0, in1,    // multiplexer inputs
    input          sel,         // select line
    output [3:0] ones, tens, hundreds
);

```

```
wire [N-1:0] mux_out;

// MUX instantiation
n_bit_mux #(N) mux_inst (
    .in0(in0), .in1(in1), .sel(sel), .out(mux_out)
);

// BCD converter instantiation
binary_to_bcd #(N) bcd_inst (
    .binary(mux_out),
    .ones(ones), .tens(tens), .hundreds(hundreds)
);

endmodule
```

6.6 TESTBENCH CODE

```
//=====
// Testbench
//=====

module tb_mux_with_bcd;

    parameter N = 8;

    reg [N-1:0] in0, in1;
    reg        sel;
    wire [3:0] ones, tens, hundreds;

    // DUT instantiation
    mux_with_bcd #(N) dut (
        .in0(in0), .in1(in1), .sel(sel),
```

```
        .ones(ones), .tens(tens), .hundreds(hundreds)
    );

initial begin

    // waveform dump (for simulation, optional in Vivado)
    $dumpfile("mux_bcd.vcd");
    $dumpvars(0, tb_mux_with_bcd);

    // test cases
    in0 = 8'd25;   in1 = 8'd123; sel = 0; #10; // expect 025
    sel = 1;                                     #10; // expect 123

    in0 = 8'd99;   in1 = 8'd200; sel = 0; #10; // expect 099
    sel = 1;                                     #10; // expect 200

    in0 = 8'd45;   in1 = 8'd7;   sel = 0; #10; // expect 045
    sel = 1;                                     #10; // expect 007

    $finish;
end

endmodule
```

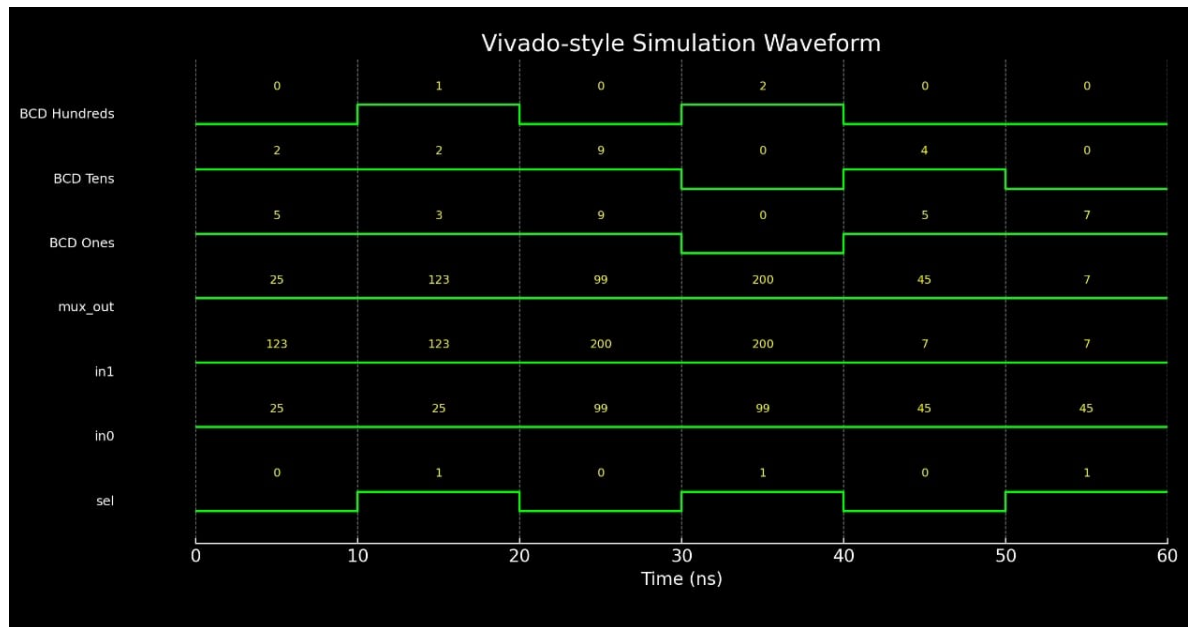


Figure 6.4: OUTPUT WAVEFORMS



CONCLUSION

The out represent the output of the multiplication of two numbers. A value of 1 in finish signal indicates that the multiplication is complete while a value of 0 indicates that the multiplication is ongoing. The clk and reset are input signals used to set the clock and reset the circuit to initial state respectively. The bcd represents the binary-coded decimal representation of the output. The a in represents the input multiplicand. The b in represents the input multiplier. A value of 0 in start loads the initial value into the registers, while a value of 1 performs the multiplication.

8

REFERENCES

- [1] *Rajeev Madavan, verilog HDL reference guide, Automata Publishing company, CA, 1993. ISBN 0-9627488-4-6.*
- [2] *Stuart Sutherland, Verilog HDL 2.0 Language Reference Guide, Stuartland Consulting, CO, 1993.*
- [3] *Verilog HDL : A Guide to Digital Design and Synthesis, Samir Palnitkar, Prentice Hall 1996, ISBN: 0134516753.*
- [4] *Bergeron, Janick, Cerny, Eduard, Hunter, Alan, and Nightingale, Andrew. Verification Methodology Manual for SystemVerilog. Norwell, MA: Springer, 2005.*
- [5] *Vijayaraghavan, Srikanth, and Ramanathan, Meyyappan. A Practical Guide for SystemVerilog Assertions. Norwell, MA: Springer, 2005 .*
- [6] *A Verilog HDL Primer, Second Edition, J. Bhasker, 1999, Star Galaxy Publishing, ISBN 0- 9650391-7-X*



Figure 8.1: CERTIFICATE